

Air University



DSA (Lab)

Year 2025

Mr. Ali Raza

Academic Year: 2024 – 2028

Department of Cyber Security

USAMA ARSHAD | 247141

BS – Cyber Security – Fall – 24 (A)

Date of Submission: October 28, 2025

Assignment # 02

Part A – Stack

1. Implement a Stack using Array.

Code

```
// Implementing Stack using Array
#include <iostream>
using namespace std;

class Stack
{
    int array[5];
    int top;

public:
    Stack() { top = -1; } // Constructor

    void push() // Push method: Add an element to the top of the stack
    {
        int val;
        cout << "Enter Value to Push: ";
        cin >> val;

        if (top >= 4) // Check for stack overflow (array full)
        {
            cout << "Stack Overflow!" << endl;
            return;
        }
        top++; // Increment top and assign value
        array[top] = val;
        cout << "Pushed: " << val << endl;
    }

    int pop() // Pop method: Remove and return the top element
    {
        if (top < 0) // Check for stack underflow (empty stack)
        {
            cout << "Stack Underflow!" << endl;
            return -1;
        }

        int popVal;
        // Retrieve and decrement top
        popVal = array[top--];
        cout << "Popped: " << popVal << endl;
        return popVal;
    }

    int peek() // Peek method: View the top element without removing it
    {
        if (top < 0) // Check if stack is empty
        {
```

```
        cout << "Stack is Empty." << endl;
        return -1;
    }
    cout << "Top Element: " << array[top] << endl;
    return array[top];
}

void display() // Display method: Print all elements from top to bottom
{
    if (top < 0) // Check if stack is empty
    {
        cout << "Stack is Empty." << endl;
        return;
    }
    cout << "Stack (Top -> Bottom): ";

    for (int i = top; i >= 0; i--) // Loop from top to bottom to display elements
    {
        cout << array[i];
        if (i != 0)
        {
            cout << " -> ";
        }
    }
    cout << endl;
}

};

int main()
{
    Stack s;
    int choice;

    cout << "\n-: Stack Implementation using Array :-" << endl;

    do
    {
        cout << "\nMenu:" << endl;
        cout << "1. PUSH (Insert an element)" << endl;
        cout << "2. POP (Delete an element)" << endl;
        cout << "3. PEEK (View the top element)" << endl;
        cout << "4. DISPLAY (Show all elements)" << endl;
        cout << "5. Exit" << endl;

        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice)
        {
            case 1:
                s.push();
                break;
            case 2:
                s.pop();
                break;
            case 3:
                s.peek();
                break;
```

```
case 4:
    s.display();
    break;
case 5:
    cout << "Exiting..." << endl;
    break;
default:
    cout << "Invalid choice! Please try again." << endl;
}
} while (choice != 5);

return 0;
}
```

Output

-: Stack Implementation using Array :-

Menu:

1. PUSH (Insert an element)
2. POP (Delete an element)
3. PEEK (View the top element)
4. DISPLAY (Show all elements)
5. Exit

Enter your choice: 1

Enter Value to Push: 5

Pushed: 5

Menu:

1. PUSH (Insert an element)
2. POP (Delete an element)
3. PEEK (View the top element)
4. DISPLAY (Show all elements)
5. Exit

Enter your choice: 1

Enter Value to Push: 6

Pushed: 6

Menu:

1. PUSH (Insert an element)
2. POP (Delete an element)
3. PEEK (View the top element)
4. DISPLAY (Show all elements)
5. Exit

Enter your choice: 4

Stack (Top -> Bottom): 6 -> 5

Menu:

1. PUSH (Insert an element)
2. POP (Delete an element)
3. PEEK (View the top element)
4. DISPLAY (Show all elements)
5. Exit

Enter your choice: 2

Popped: 6

```
Menu:
1. PUSH (Insert an element)
2. POP (Delete an element)
3. PEEK (View the top element)
4. DISPLAY (Show all elements)
5. Exit
Enter your choice: 3
Top Element: 5

Menu:
1. PUSH (Insert an element)
2. POP (Delete an element)
3. PEEK (View the top element)
4. DISPLAY (Show all elements)
5. Exit
Enter your choice: 5
Exiting...
```

2. Implement a Stack using Linked List.

Code

```
// implementing Stack using Linked List
#include <iostream>
using namespace std;

class Node // Node class for linked list elements
{
public:
    int data;
    Node *next;

    Node(int d) : data(d), next(nullptr) {}
};

class Stack // Stack class implementing LIFO using linked list
{
private:
    Node *TopNode;

public:
    Stack() { TopNode = nullptr; } // Constructor: Initialize top to null

    void push(int val) // Push: Add element to top of stack
    {
        Node *newNode = new Node(val);
        if (TopNode == nullptr)
        {
            TopNode = newNode;
        }
        else
        {
            newNode->next = TopNode;
        }
    }
};
```

```
        TopNode = newNode;
    }
    cout << "Pushed: " << val << endl;
}

int pop() // Pop: Remove and return top element
{
    if (TopNode == nullptr)
    {
        cout << "Stack is Empty. Cannot pop." << endl;
        return -1;
    }

    int value = TopNode->data;
    Node *temp = TopNode;
    TopNode = TopNode->next;
    delete temp;
    cout << "Popped: " << value << endl;
    return value;
}

int peek() // Peek: View top element without removing
{
    if (TopNode == nullptr)
    {
        cout << "Stack is Empty. No top element." << endl;
        return -1;
    }

    cout << "\nTop element: " << TopNode->data << endl;
    return TopNode->data;
}

void display() // Display: Show all elements from top to bottom
{
    if (TopNode == nullptr)
    {
        cout << "The Stack is Empty." << endl;
        return;
    }

    cout << "\nStack (Top -> Bottom): ";
    Node *current = TopNode;
    while (current != nullptr)
    {
        cout << current->data;
        if (current->next != nullptr)
        {
            cout << " -> ";
        }
        current = current->next;
    }
    cout << endl;
}

~Stack() // Destructor: Clean up memory
{
    while (TopNode != nullptr)
```

```
{
    Node *temp = TopNode;
    TopNode = TopNode->next;
    delete temp;
}
}
};

int main()
{
    cout << endl;
    Stack s1;
    int choice, value;

    cout << "\n-: Stack Implementation using Linked List :-" << endl;
    do
    {
        cout << "\nMenu:" << endl;
        cout << "1. PUSH (Insert an element)" << endl;
        cout << "2. POP (Delete an element)" << endl;
        cout << "3. PEEK (View the top element)" << endl;
        cout << "4. DISPLAY (Show all elements)" << endl;
        cout << "5. Exit" << endl;

        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice)
        {
            case 1:
                cout << "Enter value to push: ";
                cin >> value;
                s1.push(value);
                break;
            case 2:
                s1.pop();
                break;
            case 3:
                s1.peek();
                break;
            case 4:
                s1.display();
                break;
            case 5:
                cout << "Exiting..." << endl;
                break;
            default:
                cout << "Invalid choice. Please try again." << endl;
        }
    } while (choice != 5);

    cout << endl;
    return 0;
}
```

Output

```
-: Stack Implementation using Linked List :-
```

```
Menu:
```

1. PUSH (Insert an element)
2. POP (Delete an element)
3. PEEK (View the top element)
4. DISPLAY (Show all elements)
5. Exit

```
Enter your choice: 1
```

```
Enter value to push: 3
```

```
Pushed: 3
```

```
Menu:
```

1. PUSH (Insert an element)
2. POP (Delete an element)
3. PEEK (View the top element)
4. DISPLAY (Show all elements)
5. Exit

```
Enter your choice: 1
```

```
Enter value to push: 4
```

```
Pushed: 4
```

```
Menu:
```

1. PUSH (Insert an element)
2. POP (Delete an element)
3. PEEK (View the top element)
4. DISPLAY (Show all elements)
5. Exit

```
Enter your choice: 4
```

```
Stack (Top -> Bottom): 4 -> 3
```

```
Menu:
```

1. PUSH (Insert an element)
2. POP (Delete an element)
3. PEEK (View the top element)
4. DISPLAY (Show all elements)
5. Exit

```
Enter your choice: 3
```

```
Top element: 4
```



```
Menu:
1. PUSH (Insert an element)
2. POP (Delete an element)
3. PEEK (View the top element)
4. DISPLAY (Show all elements)
5. Exit
Enter your choice: 2
Popped: 4

Menu:
1. PUSH (Insert an element)
2. POP (Delete an element)
3. PEEK (View the top element)
4. DISPLAY (Show all elements)
5. Exit
Enter your choice: 5
Exiting...
```

Part B – Queue

1. Implement a Queue using Array

Code

```
// Implementing Simple Queue using Array
#include <iostream>
using namespace std;

class Queue
{
private:
    static const int num = 5;
    int array[num];
    int front, rear;

public:
    Queue() : front(0), rear(-1) {} // Constructor

    bool isEmpty() // Check if queue is empty
    {
        return front > rear; // Empty if front exceeds rear
    }

    bool isFull() // Check if queue is full
    {
        return rear == num - 1; // Full if rear at last index
    }

    void enqueue() // Add an element to the rear of the queue
```

```
{
    int val;
    cout << "Enter value to Insert: ";
    cin >> val;

    if (isFull()) // Check if queue is full
    {
        cout << "Queue is full." << endl;
        return;
    }
    // Increment rear and insert value
    rear++;
    array[rear] = val;
    cout << "Value " << val << " Added." << endl;
}

void dequeue() // Remove an element from the front of the queue
{
    if (isEmpty()) // Check if queue is empty
    {
        cout << "Queue is empty." << endl;
        return;
    }
    // Output dequeued value and increment front
    cout << "Value " << array[front] << " Removed." << endl;
    front++;
}

void display() // Print all elements from front to rear
{
    if (isEmpty()) // Check if queue is empty
    {
        cout << "Queue is empty." << endl;
        return;
    }
    cout << "Queue elements (Front to Rear): ";

    for (int i = front; i <= rear; i++) // Display elements from front to rear
    {
        cout << array[i];
        if (i < rear)
        {
            cout << " -> ";
        }
    }
    cout << endl;
}

};

int main()
{
    Queue q;
    int choice, value;

    cout << "\n-: Simple Queue Implementation using Array :-" << endl;

    do
    {
```

```
cout << "\nMenu:" << endl;
cout << "1. Enqueue (Insert an element)" << endl;
cout << "2. Dequeue (Delete an element)" << endl;
cout << "3. Display (Show all elements)" << endl;
cout << "4. Exit" << endl;
cout << "Enter your choice: ";
cin >> choice;

switch (choice)
{
case 1:
    q.enqueue();
    break;
case 2:
    q.dequeue();
    break;
case 3:
    q.display();
    break;
case 4:
    cout << "Exiting..." << endl;
    break;
default:
    cout << "Invalid choice! Please try again." << endl;
}
} while (choice != 4);

return 0;
}
```

Output

-: Simple Queue Implementation using Array :-

Menu:

1. Enqueue (Insert an element)
2. Dequeue (Delete an element)
3. Display (Show all elements)
4. Exit

Enter your choice: 1

Enter value to Insert: 7

Value 7 Added.

Menu:

1. Enqueue (Insert an element)
2. Dequeue (Delete an element)
3. Display (Show all elements)
4. Exit

Enter your choice: 1

Enter value to Insert: 8

Value 8 Added.

Menu:

1. Enqueue (Insert an element)
2. Dequeue (Delete an element)
3. Display (Show all elements)
4. Exit

Enter your choice: 3

Queue elements (Front to Rear): 7 -> 8

Menu:

1. Enqueue (Insert an element)
2. Dequeue (Delete an element)
3. Display (Show all elements)
4. Exit

Enter your choice: 2

Value 7 Removed.

Menu:

1. Enqueue (Insert an element)
2. Dequeue (Delete an element)
3. Display (Show all elements)
4. Exit

Enter your choice: 3

Queue elements (Front to Rear): 8

Menu:

1. Enqueue (Insert an element)
2. Dequeue (Delete an element)
3. Display (Show all elements)
4. Exit

Enter your choice: 4

Exiting...

2. Implement Queue using Linked List

Code

```
// Implementing Queue using Linked List
#include <iostream>
using namespace std;

class Node
{
public:
    int data;
    Node *next;

    Node(int d) : data(d), next(nullptr) {} // Constructor:
};
```

```
class Queue
{
private:
    Node *front;
    Node *rear;

public:
    Queue() : front(nullptr), rear(nullptr) {} // Constructor:

    bool isEmpty() // Check if queue is empty
    {
        return front == nullptr;
    }

    void enqueue() // Add an element to the rear of the queue
    {
        int val;
        cout << "Enter Value to Insert: ";
        cin >> val;

        Node *newNode = new Node(val);

        if (isEmpty()) // If queue is empty, set front and rear to new node
        {
            front = rear = newNode;
        }
        else
        {
            // Link new node to rear and update rear
            rear->next = newNode;
            rear = newNode;
        }
        cout << "Value " << val << " Added." << endl;
    }

    void dequeue() // Remove an element from the front of the queue
    {
        if (isEmpty()) // Check if queue is empty
        {
            cout << "Queue is Empty." << endl;
            return;
        }

        Node *temp = front;
        int val = front->data;
        // Move front to next node
        front = front->next;
        delete temp; // Free memory

        cout << "Value " << val << " Removed." << endl;
    }

    void display() // Print all elements from front to rear
    {
        if (isEmpty()) // Check if queue is empty
        {
            cout << "Queue is Empty." << endl;
        }
    }
}
```

```
        return;
    }

    Node *current = front;
    cout << "Values (Front to Rear): ";
    // Loop through the queue to display elements
    while (current)
    {
        cout << current->data;
        if (current->next)
        {
            cout << " -> ";
        }
        current = current->next;
    }
    cout << endl;
}

~Queue() // Clean up memory by dequeuing all elements
{
    // Dequeue until queue is empty
    while (!isEmpty())
    {
        dequeue();
    }
}

};

int main()
{
    Queue q;
    int choice;

    cout << "\n-: Queue Implementation using Linked List :-" << endl;

    do
    {
        cout << "\nMenu:" << endl;
        cout << "1. Enqueue (Insert an element)" << endl;
        cout << "2. Dequeue (Delete an element)" << endl;
        cout << "3. DISPLAY (Show all elements)" << endl;
        cout << "4. Exit" << endl;

        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice)
        {
            case 1:
                q.enqueue();
                break;
            case 2:
                q.dequeue();
                break;
            case 3:
                q.display();
                break;
            case 4:
```

```
        cout << "Exiting..." << endl;
        break;
    default:
        cout << "Invalid choice! Please try again." << endl;
    }
} while (choice != 4);

return 0;
}
```

Output

-: Queue Implementation using Linked List :-

Menu:

1. Enqueue (Insert an element)
2. Dequeue (Delete an element)
3. DISPLAY (Show all elements)
4. Exit

Enter your choice: 1

Enter Value to Insert: 3

Value 3 Added.

Menu:

1. Enqueue (Insert an element)
2. Dequeue (Delete an element)
3. DISPLAY (Show all elements)
4. Exit

Enter your choice: 1

Enter Value to Insert: 4

Value 4 Added.

Menu:

1. Enqueue (Insert an element)
2. Dequeue (Delete an element)
3. DISPLAY (Show all elements)
4. Exit

Enter your choice: 3

Values (Front to Rear): 3 -> 4

Menu:

1. Enqueue (Insert an element)
2. Dequeue (Delete an element)
3. DISPLAY (Show all elements)
4. Exit

Enter your choice: 2

Value 3 Removed.

```
Menu:
1. Enqueue (Insert an element)
2. Dequeue (Delete an element)
3. DISPLAY (Show all elements)
4. Exit
Enter your choice: 3
Values (Front to Rear): 4

Menu:
1. Enqueue (Insert an element)
2. Dequeue (Delete an element)
3. DISPLAY (Show all elements)
4. Exit
Enter your choice: 4
Exiting...
Value 4 Removed.
```

3. Implement a Circular Queue using Array

Code

```
// Implementing Queue using Array (Circular Queue)

#include <iostream>
using namespace std;

class Queue
{
private:
    static const int num = 5;
    int array[num];
    int front, rear;

public:
    Queue() // Constructor
    {
        front = -1;
        rear = -1;
    }

    bool isEmpty() // Check if queue is empty
    {
        return front == -1;
    }

    bool isFull() // Check if queue is full using circular logic
    {
        return (rear + 1) % num == front;
    }

    void enqueue() // Add an element to the rear of the queue
    {
        int val;
        cout << "Enter Value to Insert: ";
```



```
    cin >> val;

    if (isFull()) // Check if queue is full
    {
        cout << "Queue is Full." << endl;
        return;
    }
    if (isEmpty()) // If queue is empty, set front to 0
    {
        front = 0;
    }

    // Move rear circularly and insert value
    rear = (rear + 1) % num;
    array[rear] = val;
    cout << "Value " << val << " Added." << endl;
}

void dequeue() // Remove an element from the front of the queue
{
    if (isEmpty()) // Check if queue is empty
    {
        cout << "Queue is Empty." << endl;
        return;
    }

    cout << "Value " << array[front] << " Removed." << endl;

    if (front == rear) // If only one element, reset to empty
    {
        front = rear = -1;
    }
    else
    {
        // Move front circularly
        front = (front + 1) % num;
    }
}

void display() // Print all elements from front to rear
{
    if (isEmpty()) // Check if queue is empty
    {
        cout << "Queue is Empty." << endl;
        return;
    }

    cout << "Values (Front to Rear): ";
    int i = front;

    while (true) // Loop through the circular queue
    {
        cout << array[i];
        if (i == rear) // Break if reached rear
        {
            break;
        }
        cout << " -> ";
    }
}
```

```
        // Move to next index circularly
        i = (i + 1) % num;
    }
    cout << endl;
}
};

int main()
{
    Queue q;
    int choice;
    cout << "\n-: Circular Queue Implementation using Array :-" << endl;

    do
    {
        cout << "\nMenu:" << endl;
        cout << "1. Enqueue (Insert an element)" << endl;
        cout << "2. Dequeue (Delete an element)" << endl;
        cout << "3. Display (Show all elements)" << endl;
        cout << "4. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice)
        {
            case 1:
                q.enqueue();
                break;
            case 2:
                q.dequeue();
                break;
            case 3:
                q.display();
                break;
            case 4:
                cout << "Exiting..." << endl;
                break;
            default:
                cout << "Invalid choice! Please try again." << endl;
        }
    } while (choice != 4);
    return 0;
}
```

Output

```
-: Circular Queue Implementation using Array :-
```

```
Menu:
```

1. Enqueue (Insert an element)
2. Dequeue (Delete an element)
3. Display (Show all elements)
4. Exit

```
Enter your choice: 1
```

```
Enter Value to Insert: 4
```

```
Value 4 Added.
```

```
Menu:
```

1. Enqueue (Insert an element)
2. Dequeue (Delete an element)
3. Display (Show all elements)
4. Exit

```
Enter your choice: 1
```

```
Enter Value to Insert: 3
```

```
Value 3 Added.
```

```
Menu:
```

1. Enqueue (Insert an element)
2. Dequeue (Delete an element)
3. Display (Show all elements)
4. Exit

```
Enter your choice: 3
```

```
Values (Front to Rear): 4 -> 3
```

```
Menu:
```

1. Enqueue (Insert an element)
2. Dequeue (Delete an element)
3. Display (Show all elements)
4. Exit

```
Enter your choice: 2
```

```
Value 4 Removed.
```

```
Menu:  
1. Enqueue (Insert an element)  
2. Dequeue (Delete an element)  
3. Display (Show all elements)  
4. Exit
```

```
Enter your choice: 3
```

```
Values (Front to Rear): 3
```

```
Menu:  
1. Enqueue (Insert an element)  
2. Dequeue (Delete an element)  
3. Display (Show all elements)  
4. Exit
```

```
Enter your choice: 4
```

```
Exiting...
```

THE END
